Enablement Kit — Microsoft Defender for Cloud Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Enablement Kit: Microsoft Defender for Cloud Apps 4
- What you get . . . 4
- What this delivers . . . 4
- The standard deployment model . . . 4
- Boundary . . . 5
- Effort . . . 5

1. Solution Overview — Enablement Kit: Microsoft Defender for Cloud Apps 5
- What this enablement kit is . . . 5
- What Microsoft Defender for Cloud Apps is (the concept) . . . 5
- How the operator uses it (the console and workflows) . . . 6
- The standard deployment model (how the enablement kit is run) . . . 6
- The five labs (scope of the hands-on) . . . 6
- What is in scope . . . 7
- What is out of scope (and where it lives) . . . 7
- Connections and authentication used . . . 7

2. Requirements — Enablement Kit: Microsoft Defender for Cloud Apps 7
- R1. Defender for Cloud Apps licensed and active . . . 7
- R2. A Cloud Discovery data source . . . 8
- R3. At least one connected app (API connector) . . . 8
- R4. App governance enabled (for the OAuth/app-behavior labs) . . . 8
- R5. Advanced hunting access (CloudAppEvents) . . . 8
- R6. Read-only Graph permissions for the lab helpers . . . 8
- R7. Facilitator with the right role, attendees read-only . . . 9

Global Requirements — Defender (category-level) 9
- G1. Microsoft 365 / Entra tenant . . . 9
- G2. Defender licensing . . . 9
- G3. Administrative roles (least privilege) . . . 9
- G4. Onboarding reach to endpoints / identity / workloads . . . 9
- G5. Network access to Defender service endpoints . . . 10
- G6. A pilot ring . . . 10
- G7. Log Analytics workspace (where required) . . . 10
- G8. Response governance . . . 10

3. Implementation — Enablement Kit Runbook: Microsoft Defender for Cloud Apps 10
- Lab 0 — Readiness check (Assess) . . . 11
- Lab 1 — Cloud Discovery & shadow IT (Configure/learn) . . . 11

Lab 2 — App governance & connected apps (Pilot) . . . 12
Lab 3 — OAuth app risk (Pilot, approval-gated rehearsal) . . . 12
Lab 4 — Session & access policies, report-only first (Validate) . . . 12
Lab 5 — Anomaly detection & investigation (Validate/Expand) . . . 13
Wrap-up (Expand) . . . 13
5. Testing & Knowledge-Check — Enablement Kit: Microsoft Defender for Cloud Apps 14
Part A — Environment / lab validation (scripts) . . . 14
Part B — Knowledge-check (per attendee) . . . 14
Part C — Live demonstration (capstone) . . . 15
6. Troubleshooting — Enablement Kit: Microsoft Defender for Cloud Apps 15
Cloud apps section missing or blades won't open . . . 15
Cloud Discovery shows no apps (Lab 1) . . . 15
No connected apps / connector shows no activity (Lab 2) . . . 15
App governance unavailable (Labs 2/3) . . . 16
`CloudAppEvents` returns nothing (Labs 2/5) . . . 16
`Connect-MgGraph` fails or scopes not granted . . . 16
`WSMCA-OAuthApp-Lab.ps1` lists apps but no permission grants . . . 16
Session policy has no effect / can't be created (Lab 4) . . . 16
Anomaly-detection alerts absent (Lab 5) . . . 17
Portal menu paths don't match . . . 17
7. Best Practices & Where to Go Next — Enablement Kit: Microsoft Defender for Cloud Apps 17
Running the enablement kit well . . . 17
Operating MDA well (the habits to instill) . . . 17
Common anti-patterns to avoid . . . 18
Where to go next (adjacent catalog areas) . . . 18
8. Deliverables Summary — Enablement Kit: Microsoft Defender for Cloud Apps 18
Documentation (source/) . . . 18
Lab-helper assets (scripts/) . . . 19
Take-home runbook artifacts (produced during the labs) . . . 19
Appendix A — PowerShell Scripts 21

Executive Summary — Enablement Kit: Microsoft Defender for Cloud Apps

White-label · Defender · Enablement / Training deliverable

A licensed tool only pays off when the team can drive it. Many organizations turn on **Microsoft Defender for Cloud Apps (MDA)** — the cloud access security broker (CASB) in Microsoft Defender — then leave most of it unused because no one is confident with Cloud Discovery, app governance, or the policy engine. This is a **structured enablement enablement kit** that takes your admins and SOC analysts from "we have it" to "we operate it" — through concepts, **guided hands-on labs**, a **take-home runbook**, and a **knowledge-check**.

This is a training deliverable, not a deployment project. It assumes MDA is licensed and reachable — the *how-to-connect-every-app* work lives in the catalog's Cloud App DLP / onboarding solutions — and focuses on the **operating skills**: running **Cloud Discovery** to expose shadow IT, using **app governance** to watch OAuth apps and their API usage, assessing **OAuth app risk** and consent, building **session and access policies** (Conditional Access App Control), and reading **anomaly-detection** alerts. Every lab is **read-only or preview-first / report-only**, so a team can learn safely without disrupting users.

What you get

- A plain-language **Solution Overview** of Defender for Cloud Apps — what a CASB does, how Cloud Discovery, app connectors, app governance, and the policy engine work, and the console/workflows an operator uses daily.
- A **Requirements Guide** covering the enablement-kit-specific prerequisites (MDA licensing, a discovery data source, at least one connected app, app-governance enablement) with verify / implement-if-missing for each.
- An **Implementation Guide** that is the **facilitator + student runbook** — five guided labs, each with named objects, exact portal navigation, and explicit pass/fail checks.
- Five **lab-helper assets**: read-only PowerShell discovery/validation/report scripts (Microsoft Graph) plus a **KQL query pack** for advanced hunting over the `CloudAppEvents` table.
- A **Testing / knowledge-check** guide and a **Troubleshooting** guide.
- A **Best-Practices + where-to-go-next** guide that points to the adjacent catalog areas (Cloud App DLP, Conditional Access, threat hunting).

What this delivers

A team that can **confidently operate Microsoft Defender for Cloud Apps**: they can run and read a Cloud Discovery report and rank shadow-IT apps, use app governance to spot a risky OAuth app, assess and (with approval) act on OAuth consent risk, author a **report-only** session/access policy before enforcing it, and triage an anomaly-detection alert — with a **runbook they keep** and reuse.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — delivered against a **pilot** scope, each skill validated with a pass/fail gate, then the team expands the practices to the full environment.

Boundary

This enablement kit delivers the **technical enablement and the operating runbook**. Acceptable-use, data-classification, records, and app-sanctioning decisions belong to the customer's security, compliance, and business functions — the enablement kit teaches the controls, not the decision. Policies are taught **report-only / monitor-first**; blocking and session controls are **approval-gated**. Some MDA capabilities are **license-** or **preview-gated**; those facts are flagged **verify current availability in your tenant**.

Effort

A competent facilitator can run the full enablement kit in a focused one- to two-day engagement against a pilot; the labs and runbook are then reusable to onboard new analysts. No enforcing policy is required to complete the enablement kit — the labs are report-only / preview-first.

1. Solution Overview — Enablement Kit: Microsoft Defender for Cloud Apps

What this enablement kit is

This is an **enablement / training** deliverable, not a deployment project. It teaches your admins and SOC analysts to **operate Microsoft Defender for Cloud Apps (MDA)** confidently — through concept sessions, **five guided hands-on labs**, a **take-home runbook**, and a **knowledge-check**. The output is a skilled team and a reusable runbook. The labs are designed to run against a **pilot** scope and are **read-only or report-only / preview-first**, so learning never disrupts users.

What Microsoft Defender for Cloud Apps is (the concept)

Microsoft Defender for Cloud Apps is Microsoft's **cloud access security broker (CASB)** — the cloud-app visibility and control pillar of Microsoft Defender XDR. It answers four questions:

- **What cloud apps are in use? Cloud Discovery** ingests traffic logs (from a network appliance, from Defender for Endpoint, or uploaded) and builds a catalog of **discovered apps** with a **risk score**, exposing **shadow IT**.
- **What is happening inside sanctioned apps? App connectors** (API-based, for apps like Microsoft 365, Salesforce, Box, ServiceNow, etc.) pull activity, files, and accounts so you can apply policies and detect anomalies.
- **Which OAuth apps have access, and are they risky? App governance** and the **OAuth apps** inventory show third-party apps granted access to your data, their permissions, and behavior, with risk indicators.
- **Can I control a session in real time? Conditional Access App Control** (session/access policies, integrated with Entra Conditional Access) can monitor or restrict actions like download, copy, and print inside a session.

On top of these, MDA runs **anomaly-detection** policies (impossible travel, mass download, unusual admin activity, ransomware-like activity) and **threat-detection** policies, surfacing alerts into Defender XDR incidents and the `CloudAppEvents` advanced-hunting table.

How the operator uses it (the console and workflows)

1. **The unified portal** — `https://security.microsoft.com` → **Cloud apps** section. (Some deep MDA settings still route through the classic portal / are being consolidated — *verify current navigation in your tenant*.)

2. **Cloud Discovery** — dashboards, discovered apps, app risk scores, and the ability to tag apps **Sanctioned / Unsanctioned**.

3. **Connected apps & app governance** — the API connectors and the OAuth app inventory with risk and usage.

4. **Policies** — activity, file, session, access, anomaly-detection, and OAuth app policies. New policies are authored **report-only / monitor** first.

5. **Alerts & advanced hunting** — MDA alerts appear in **Incidents & alerts**; `CloudAppEvents` is queryable in **Advanced hunting**.

The standard deployment model (how the enablement kit is run)

• **Assess** — confirm the pilot scope, MDA licensing, a discovery data source, and at least one connected app (Lab 0 readiness).

• **Configure** — set up the enablement kit objects: the Cloud Discovery review, the saved hunting queries, and a **report-only** session policy.

• **Pilot** — run the five labs against the pilot scope.

• **Validate** — each lab has a **pass/fail gate**; the enablement kit ends with a **knowledge-check**.

• **Expand** — apply the runbook and policies to the full environment through the customer's change process.

The five labs (scope of the hands-on)

• **Lab 1 — Cloud Discovery & shadow IT.** Run/read a discovery report; rank apps by risk; tag Sanctioned/Unsanctioned.

• **Lab 2 — App governance & connected apps.** Explore a connected app's activity; use app governance to review app behavior and API usage.

• **Lab 3 — OAuth app risk.** Inventory OAuth apps, read permissions and risk, and rehearse the **approval-gated** disable/revoke of a risky app.

• **Lab 4 — Session & access policies (report-only first).** Author a Conditional Access App Control session policy in **monitor/report-only** mode; understand the enforce path.

• **Lab 5 — Anomaly detection & investigation.** Read anomaly-detection alerts, triage with the entity/activity log and advanced hunting, and rehearse response.

What is in scope

- Concept enablement, five guided labs, the take-home runbook, and the knowledge-check for **operating** MDA.
- Read-only discovery/validation/reporting helpers and a KQL hunting pack for the labs.

What is out of scope (and where it lives)

- **Connecting every app / building the full DLP program** — see the catalog's **Cloud App DLP** and onboarding solutions.
- **Conditional Access design** — see the **Identity / Zero Trust / Conditional Access** solutions; MDA session control integrates with CA.
- **Broad cross-pillar threat hunting and SOC process** — see the **Threat Hunting** and **SOC / XDR operations** solutions.
- **Acceptable-use, app-sanctioning, records, and data-classification decisions** — the customer's security/compliance/business functions'; the enablement kit teaches the technical workflow only.

Connections and authentication used

- The **Microsoft Defender portal** (`https://security.microsoft.com` → Cloud apps) for discovery, connected apps, app governance, policies, alerts, and advanced hunting.
- **Microsoft Graph** (via the lab PowerShell helpers, delegated **read** scopes such as `SecurityAlert.Read.All`, `Application.Read.All`, `Directory.Read.All`) for read-only discovery, validation, and reporting.
- No lab enforces a policy; session/OAuth actions are **report-only / approval-gated**.

2. Requirements — Enablement Kit: Microsoft Defender for Cloud Apps

The **category-level prerequisites** (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, Log Analytics, response governance) are covered once in `global-requirements-defender.md` — read that first. This section lists only the **enablement-kit-specific** prerequisites. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Defender for Cloud Apps licensed and active

Description: MDA is licensed (standalone or via E5 Security / E5 Compliance bundles) and the **Cloud apps** area is reachable in the Defender portal. **Why required:** Every lab uses the MDA console. Without the license the Cloud apps blades, connectors, and policies are unavailable. **How to verify:** In `https://security.microsoft.com`, confirm the **Cloud apps** section is present and opens. `WSMCA-Validate.ps1` checks that Graph returns cloud-app data. *Confirm the exact plan in Admin center → Billing → Licenses — verify current availability.* **How to implement if missing:** Assign the MDA plan (often bundled in E5 Security) before running the enablement kit.

R2. A Cloud Discovery data source

Description: A source of traffic for **Cloud Discovery** — a continuous feed from **Defender for Endpoint**, a network appliance log, or a manually uploaded traffic log (snapshot report). **Why required:** Lab 1 reads a discovery report. With no data source there are no discovered apps to rank. **How to verify:** In **Cloud apps** → **Cloud Discovery**, confirm a dashboard with discovered apps exists, or that the Defender for Endpoint integration is on. `WSMCA-Discovery.ps1` reports whether discovery data is present. **How to implement if missing:** Turn on the **Defender for Endpoint** → **MDA** integration (fastest), or upload a sample traffic log to produce a **snapshot report** for the lab. *Verify current integration steps in your tenant.*

R3. At least one connected app (API connector)

Description: At least one **app connector** (e.g. Microsoft 365, or another supported SaaS app) is connected via API. **Why required:** Lab 2 explores a connected app's activity and app-governance signal. Without a connector there is no in-app activity to examine. **How to verify:** In **Cloud apps** → **Connected apps** (App connectors), confirm at least one connected app with recent activity. `WSMCA-Validate.ps1` flags connector presence via available signal. **How to implement if missing:** Connect the **Microsoft 365** connector at minimum (native, quick). Additional SaaS connectors can be added later per the Cloud App DLP solution.

R4. App governance enabled (for the OAuth/app-behavior labs)

Description: App governance (the app-behavior monitoring capability within MDA) is turned on. **Why required:** Labs 2 and 3 use app governance and the OAuth app inventory to review third-party app behavior and risk. **How to verify:** In **Cloud apps** → **App governance**, confirm it is enabled and shows app data. *App governance availability/plan — verify current in your tenant.* **How to implement if missing:** Enable app governance in the MDA settings (may require the appropriate plan). If it cannot be enabled for the enablement kit, Lab 3 can still use the **OAuth apps** inventory (Enterprise applications / MDA OAuth apps) read-only.

R5. Advanced hunting access (CloudAppEvents)

Description: Attendees can open **Advanced hunting** and query the `CloudAppEvents` table. **Why required:** Labs 2 and 5 run the KQL pack (`WSMCA-LabQueries.kql`) against `CloudAppEvents` and related tables. **How to verify:** Open **Hunting** → **Advanced hunting**; run `CloudAppEvents | take 5`; confirm rows return. **How to implement if missing:** Grant attendees a role including advanced-hunting read (e.g. **Security Reader** plus hunting). *Confirm role mapping in your tenant.*

R6. Read-only Graph permissions for the lab helpers

Description: The PowerShell lab helpers authenticate to Microsoft Graph with **read-only** scopes. **Why required:** `WSMCA-Discovery.ps1`, `WSMCA-Validate.ps1`, `WSMCA-Report.ps1`, and `WSMCA-OAuthApp-Lab.ps1` read alerts, service principals, and OAuth grants. **How to verify:** `Connect-MgGraph -Scopes "SecurityAlert.Read.All","Application.Read.All","Directory.Read.All"` consents successfully. **How to implement if missing:** Have a Global Administrator consent the delegated read scopes once, or register a read-only enablement kit app. Never grant write/consent-management scopes

for the labs.

R7. Facilitator with the right role, attendees read-only

Description: One **facilitator** with an MDA administrative role (to create the report-only policy); **attendees** with **Security Reader** (plus hunting) to follow along read-only. **Why required:** Least privilege — only the facilitator authors the single **report-only** session policy; everyone else learns read-only and rehearses enforce/disable in **simulate** mode. **How to verify:** Confirm the facilitator holds the MDA admin / Security Administrator role and attendees hold Security Reader. **How to implement if missing:** Assign roles before the session; reserve enforce/disable actions for the approval-gated rehearsal only.

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud.

Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Enablement Kit Runbook: Microsoft Defender for Cloud Apps

This is the **facilitator + student runbook**. Run the labs in order against the **pilot** scope from R2/R3. Every object uses the naming convention `<Purpose> – <Scope> – <Stage>` and is reused by name in later labs, testing, and the scripts. All steps are **read-only or report-only / preview-first**; the one configuration action (a **report-only** session policy) and any disable/enforce are **facilitator-driven and approval-gated**.

Naming convention used throughout

- Report-only session policy: `Monitor-Downloads – CloudApps Lab – Pilot`.

• Saved hunting queries: `Discovery-ShadowIT – CloudApps Lab – Pilot`, `OAuthApps – CloudApps Lab – Pilot`, `Anomaly-Investigation – CloudApps Lab – Pilot`.

• Shadow-IT shortlist output: `ShadowIT-Shortlist – CloudApps Lab – Pilot`.

• OAuth risk shortlist: `OAuthRisk-Shortlist – CloudApps Lab – Pilot`.

• Investigation doc: `Investigation – CloudApps Lab – Pilot`.

*Navigation reflects the current Defender portal (`https://security.microsoft.com` → **Cloud apps**). Some MDA settings are still consolidating from the classic portal — *verify current navigation in your tenant*; the troubleshooting guide lists fallbacks.*

Lab 0 — Readiness check (Assess)

1. Sign in as the **facilitator** and open the **Cloud apps** section.

2. Confirm: a **Cloud Discovery** dashboard with data (R2), at least one **Connected app** (R3), and **App governance** enabled (R4).

3. Run `powershell -ExecutionPolicy Bypass -File .\WSMCA-Discovery.ps1` — confirm it reports MDA alerts, connected-app signal, and an OAuth app count.

4. Run `powershell -ExecutionPolicy Bypass -File .\WSMCA-Validate.ps1` — confirm readiness checks pass.

5. **Pass gate:** discovery + validate green; Cloud Discovery, a connector, and app governance all visible. Resolve via R1–R6 if not.

Lab 1 — Cloud Discovery & shadow IT (Configure/learn)

Goal: run/read a discovery report, rank apps by risk, and tag Sanctioned/Unsanctioned.

1. Open **Cloud apps** → **Cloud Discovery** → **Dashboard**. Review total apps, traffic, and top apps by users/traffic.

2. Open **Discovered apps**. Sort by **Risk score**. Discuss what drives the score (compliance, security, legal factors in the app catalog).

3. Pick 3–5 high-risk, high-usage apps. For a clearly unsanctioned one, use the **tag** action to mark **Unsanctioned** (facilitator; discuss that tagging alone is informational unless a block is wired via a proxy/appliance). Mark an approved one **Sanctioned**.

4. In **Advanced hunting**, paste the **Discovery / shadow-IT** section of `WSMCA-LabQueries.kql`; run it; **Save as** `Discovery-ShadowIT – CloudApps Lab – Pilot`.

5. Build the `ShadowIT-Shortlist – CloudApps Lab – Pilot` — the top risky apps and a recommended disposition (sanction / restrict / block-later). **App-sanctioning decisions are the customer's** — the shortlist is a recommendation.

6. **Pass gate:** each attendee can explain the risk score and name the top shadow-IT app and its proposed disposition.

Lab 2 — App governance & connected apps (Pilot)

Goal: examine a connected app's activity and use app governance to read app behavior.

1. Open **Cloud apps** → **Connected apps**; open the **Microsoft 365** (or other) connector; review **Activity log**, **Files**, and **Accounts** tabs.

2. Open **Cloud apps** → **App governance**. Review the app inventory: permissions, data access, usage, and any **risk factors / alerts** on apps.

3. Pick one app with notable permissions/usage; walk its behavior (API calls, data accessed, users). Discuss what "normal" vs "suspicious" looks like.

4. In **Advanced hunting**, run the **Connected-app activity** section of `WSMCA-LabQueries.kql` against `CloudAppEvents`; review results.

5. **Pass gate:** each attendee can navigate a connected app's activity log and state one app-governance risk indicator and what it means.

Lab 3 — OAuth app risk (Pilot, approval-gated rehearsal)

Goal: inventory OAuth apps, read consent/permission risk, and rehearse disable/revoke.

1. Open the **OAuth apps** inventory (**Cloud apps** → **OAuth apps**, and/or **App governance**). Review third-party apps that have been granted access, their **permission level**, **community use**, and **publisher verification**.

2. Run `powershell -ExecutionPolicy Bypass -File .\WSMCA-OAuthApp-Lab.ps1` — a **read-only** enumeration of service principals and OAuth grants with risk hints (high-privilege scopes, unverified publisher, broad consent). It changes nothing.

3. In **Advanced hunting**, run the **OAuth apps** section of `WSMCA-LabQueries.kql`; **Save as** `OAuthApps – CloudApps Lab – Pilot`.

4. Build the `OAuthRisk-Shortlist – CloudApps Lab – Pilot` — the riskiest apps by consent scope and verification status.

5. **Response rehearsal (approval-gated):** discuss how you would **disable / revoke consent** for a risky app (from the OAuth apps inventory or Entra Enterprise applications). **In the enablement kit, describe/queue for approval; do not revoke a production app** — revoking consent can break a legitimate integration. Revocation is a change the customer's app owners/security approve.

6. **Pass gate:** each attendee names two OAuth risk indicators and states who must approve a consent revocation.

Lab 4 — Session & access policies, report-only first (Validate)

Goal: author a Conditional Access App Control session policy in **monitor / report-only** mode and understand the enforce path.

1. Confirm the app is routed through **Conditional Access App Control** (session control requires the Entra Conditional Access integration — *verify current setup in your tenant*).

2. Open **Cloud apps** → **Policies** → **Policy management** → **Create policy** → **Session policy**.

3. Create `Monitor-Downloads – CloudApps Lab – Pilot`: session control type **Monitor only** (or an action set to **Test/report** where offered), scoped to the pilot group and the pilot app. **Do not** set it to **Block** for the enablement kit.

4. Generate benign activity (a download in the pilot session) and observe it logged — **monitored, not blocked**.

5. Discuss the enforce path: how you would change **Monitor only** → **Block download** after validation, staged to a pilot ring, with a rollback. **Enforcement is staged and change-approved — not done during training.**

6. **Pass gate:** the report-only policy exists and each attendee can describe the monitor→enforce progression and the rollback.

Lab 5 — Anomaly detection & investigation (Validate/Expand)

Goal: read anomaly-detection alerts, triage, and rehearse response.

1. Open **Incidents & alerts** → **Alerts**; filter **Service source = Defender for Cloud Apps**. Review anomaly-detection alert types (impossible travel, mass download, unusual admin activity, ransomware-like activity).

2. Take one alert (or a `CloudAppEvents` hit) through triage: which user/app, what activity, is it expected, what would escalate it.

3. In **Advanced hunting**, run the **Anomaly investigation** section of `WSMCA-LabQueries.kql`; **Save as** `Anomaly-Investigation – CloudApps Lab – Pilot`. Cross-check the user's activity across apps.

4. Capture findings in `Investigation – CloudApps Lab – Pilot`: what happened, which user/app, confidence, recommended action.

5. **Response rehearsal (approval-gated):** review MDA response options (suspend user, require sign-in, mark app unsanctioned/block via proxy). **Rehearse and queue for approval; do not action a production user. Incident-response decisions are the SOC/security team's.**

6. **Pass gate:** a completed investigation doc plus each attendee stating the fitting response and who approves it.

Wrap-up (Expand)

1. Confirm the take-home artifacts: the three saved hunting queries, the report-only session policy, the two shortlists, and the investigation doc.

2. Run the **knowledge-check** in `05-testing.md`.

3. Agree the **expand** plan: move policies from report-only to staged enforcement through the customer's change process, and use the runbook to onboard new analysts.

5. Testing & Knowledge-Check — Enablement Kit: Microsoft Defender for Cloud Apps

Testing a enablement kit means (1) confirming each lab's **pass gate** and (2) a **knowledge-check** that each attendee can operate MDA unaided. Use the lab-helper scripts by name to validate the environment; use the check-questions to validate the people.

Part A — Environment / lab validation (scripts)

A1. Readiness (before the enablement kit)

Run `WSMCA-Validate.ps1`. Expected: Graph connectivity + read scopes granted, MDA alert data reachable, connected-app signal present, advanced-hunting reminder confirmed. All green = ready to teach.

A2. Discovery sanity

Run `WSMCA-Discovery.ps1`. Expected: MDA alert count, connected-app presence, and an OAuth app count. Confirms Cloud Discovery, connectors, and OAuth data exist to teach against.

A3. OAuth inventory (after Lab 3)

Run `WSMCA-OAuthApp-Lab.ps1`; confirm it lists OAuth apps with risk hints and that the `OAuthRisk-Shortlist – CloudApps Lab – Pilot` was captured.

A4. Report-only session policy present (after Lab 4)

In **Cloud apps** → **Policies** → **Policy management**, confirm `Monitor-Downloads – CloudApps Lab – Pilot` exists and is **Monitor only** (not Block).

A5. Saved queries present (after Labs 1/3/5)

In **Advanced hunting** → **Queries**, confirm `Discovery-ShadowIT – CloudApps Lab – Pilot`, `OAuthApps – CloudApps Lab – Pilot`, and `Anomaly-Investigation – CloudApps Lab – Pilot` exist and run without error.

Part B — Knowledge-check (per attendee)

Score each attendee; passing = correct on at least 8 of 10.

1. **CASB basics:** What four questions does a CASB answer, and which MDA feature answers each?
2. **Cloud Discovery:** What feeds Cloud Discovery, and what drives an app's risk score?
3. **Shadow IT:** What does tagging an app **Unsanctioned** do, and what else is needed to actually block it?
4. **Connectors vs discovery:** How is an **app connector** different from **Cloud Discovery**?
5. **App governance:** Name two risk indicators app governance surfaces on an app.
6. **OAuth risk:** Give two OAuth app risk factors and why each is dangerous *(e.g. broad consent scopes, unverified publisher)*.

7. **Session policy:** Why author a session policy **report-only / monitor** first, and what is the enforce path?

8. **Anomaly detection:** Name three anomaly-detection alert types and one triage question for each.

9. **Hunting:** Which advanced-hunting table holds cloud-app activity, and give a field you'd pivot on. *(CloudAppEvents.)*

10. **Boundary & governance:** Which decisions (app sanctioning, consent revocation, user suspension) require the customer's approval, and who approves them?

Part C — Live demonstration (capstone)

Each attendee, unaided, on a **second** app/alert:

- Opens Discovered apps, ranks by risk, and states one app's disposition.
- Runs one saved hunting query and interprets a result.
- Names the fitting response action and the approval required.

Overall pass criteria: every attendee meets Part B (≥8/10) and completes Part C; environment checks A1–A5 pass. Record results; anyone below threshold repeats the relevant lab.

6. Troubleshooting — Enablement Kit: Microsoft Defender for Cloud Apps

Symptom → cause → fix for the common issues when running the enablement kit and its lab helpers. For category-level prerequisites (licensing, roles, network) see `global-requirements-defender.md`.

Cloud apps section missing or blades won't open

- **Symptom:** The **Cloud apps** area is absent in the Defender portal, or its blades error.
- **Cause:** MDA is not licensed/active, or the account lacks the role.
- **Fix:** Confirm the MDA plan (Admin center → Billing → Licenses; often bundled in E5 Security — *verify current availability*) and that the account holds an MDA admin / Security Reader role (R1, R7).

Cloud Discovery shows no apps (Lab 1)

- **Symptom:** The Cloud Discovery dashboard is empty; no discovered apps to rank.
- **Cause:** No discovery data source — the Defender for Endpoint integration is off and no traffic log was uploaded.
- **Fix:** Turn on the **Defender for Endpoint** → **MDA** integration (fastest continuous feed) or upload a sample traffic log to create a **snapshot report** for the lab (R2). *Verify current integration steps.*

No connected apps / connector shows no activity (Lab 2)

- **Symptom: Connected apps** is empty, or the connector shows no recent activity.

• **Cause:** No API connector added, or the connector just added (data lag).

• **Fix:** Connect the **Microsoft 365** connector at minimum (native, quick) and allow time for activity to populate (R3). Additional SaaS connectors are covered by the Cloud App DLP solution.

App governance unavailable (Labs 2/3)

• **Symptom: App governance** is greyed out or shows no data.

• **Cause:** App governance not enabled, or the plan does not include it.

• **Fix:** Enable app governance in MDA settings (*plan/availability — verify current in your tenant*). If it cannot be enabled, run Lab 3 from the **OAuth apps** inventory (Enterprise applications / MDA OAuth apps) read-only instead (R4).

`CloudAppEvents` returns nothing (Labs 2/5)

• **Symptom:** `CloudAppEvents | take 5` returns no rows.

• **Cause:** No connected apps feeding the table, warm-up lag, or missing hunting permission.

• **Fix:** Confirm a connector exists and has activity (R3); allow warm-up; confirm the attendee role includes advanced-hunting read (R5).

`Connect-MgGraph` fails or scopes not granted

• **Symptom:** Lab scripts error on connect, or `WSMCA-Validate.ps1` shows fewer scopes than requested.

• **Cause:** Read scopes not consented, or the account cannot consent.

• **Fix:** Have a Global Administrator consent the read scopes once (`SecurityAlert.Read.All`, `Application.Read.All`, `Directory.Read.All`), or register a read-only enablement kit app. Never grant write/consent-management scopes for the labs (R6).

`WSMCA-OAuthApp-Lab.ps1` lists apps but no permission grants

• **Symptom:** RiskHint is based only on publisher verification; `HighPrivScopes` is blank.

• **Cause:** `Get-MgServicePrincipalOauth2PermissionGrant` needs `Directory.Read.All` and returns delegated grants only; some app-role (application) permissions are not shown by this call.

• **Fix:** Confirm `Directory.Read.All` consent. Cross-check high-privilege **application** permissions in **Entra** → **Enterprise applications** → **Permissions**. Treat the RiskHint as a starting sort, not a verdict.

Session policy has no effect / can't be created (Lab 4)

• **Symptom:** The session policy option is missing, or a created policy does nothing.

• **Cause:** Conditional Access App Control is not set up — session control requires the Entra Conditional Access integration and the app routed through it.

• **Fix:** Confirm the CA App Control integration and that the pilot app is routed through it (*verify current setup*). Keep the policy **Monitor only** for the enablement kit; enforcement is staged later. See the

Conditional Access solution.

Anomaly-detection alerts absent (Lab 5)

• **Symptom:** No Defender-for-Cloud-Apps anomaly alerts to triage.

• **Cause:** A quiet tenant, insufficient baseline (anomaly detection learns normal behavior over time), or alerts outside the window.

• **Fix:** A quiet tenant is fine for teaching — use the runbook descriptions and the `CloudAppEvents` queries to build the investigation skill. Allow the anomaly engine a learning window. Widen the time range.

Portal menu paths don't match

• **Symptom:** A navigation path in the runbook doesn't exist.

• **Cause:** MDA settings are consolidating from the classic portal into `security.microsoft.com`; menus move.

• **Fix:** Use global search for the feature name (e.g. "Cloud Discovery", "OAuth apps", "App governance"). *Verify current navigation in your tenant* and update your copy of the runbook.

7. Best Practices & Where to Go Next — Enablement Kit: Microsoft Defender for Cloud Apps

Running the enablement kit well

• **Teach against a pilot, report-only.** Author the session policy in **monitor** mode and rehearse OAuth revocation/user suspension without executing them against production. Expand to enforcement only through the customer's change process.

• **Read-only by default.** The only configuration action is the report-only session policy (facilitator). Everything else is discovery, hunting, and reading.

• **Don't break integrations.** Revoking an OAuth app's consent can disable a legitimate business integration — always treat it as an approval-gated change with the app owner, never a live demo.

• **Use consistent object names.** The `<Purpose> – <Scope> – <Stage>` convention keeps the take-home artifacts findable and reusable. Keep it.

• **Save the artifacts.** The three saved queries, the report-only policy, the two shortlists, and the investigation doc are the take-home runbook.

Operating MDA well (the habits to instill)

• **Start with discovery, then sanction deliberately.** Rank shadow IT by risk *and* usage; a high-risk app nobody uses is lower priority than a medium-risk app everyone uses. Sanctioning is a business decision — the operator surfaces the data.

• **Watch OAuth apps continuously.** Consent grants are a common, quiet path to data. Review new/over-privileged/unverified apps on a cadence.

• **Monitor before you block.** Every session/access policy earns enforcement by proving itself in report-only first; then stage the block to a ring with a rollback.

• **Triage anomalies with context.** Impossible travel and mass-download alerts need the user's normal pattern to judge — pivot in `CloudAppEvents` before you escalate.

• **Respect the approval gate.** App sanctioning, consent revocation, and user suspension are security/business decisions. The operator produces a clean, evidenced recommendation.

• **Mind licensing and preview.** App governance and some features are plan-/preview-gated; *verify current availability in your tenant* before promising a capability.

Common anti-patterns to avoid

• Turning the enablement kit into a connect-every-app project (that is the Cloud App DLP / onboarding solution).

• Blocking an app or revoking OAuth consent live "to show it works" — you may break production.

• Enforcing a session policy before it has run report-only.

• Treating a high risk score as an automatic ban without considering business use and the approval process.

Where to go next (adjacent catalog areas)

• **Cloud App DLP** — connect more apps and build the full information-protection/DLP program on top of MDA.

• **Identity / Zero Trust / Conditional Access** — the CA integration that powers session/access control and stronger app governance.

• **Threat Hunting** — deeper cross-pillar KQL beyond `CloudAppEvents`.

• **SOC / XDR operations** — incident queue, automation, and process design that consume MDA alerts.

• **Sibling Defender enablement kits** — Identity, Endpoint, and Office 365 enablement enablement kits to round out the SOC's operating skills.

The bottom line: a team that operates MDA with confidence — exposing shadow IT, governing apps and OAuth risk, authoring report-only-first policies, and running clean, approval-gated investigations — using a runbook they keep and reuse.

8. Deliverables Summary — Enablement Kit: Microsoft Defender for Cloud Apps

This enablement kit is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation set below plus five lab-helper assets, yours to keep and reuse to train new analysts.

Documentation (source/)

• **executive-summary.md** — what the enablement kit is, the outcome, the boundary, and the effort.

• **01-solution-overview.md** — what a CASB / MDA is, how Cloud Discovery, connectors, app governance, and the policy engine work, the operator console/workflows, the five labs, and what is in/out of scope.

• **02-requirements.md** — enablement-kit-specific prerequisites (MDA licensing, a discovery data source, a connected app, app governance, hunting access, read scopes, roles), each with verify / implement-if-missing. Category-level prerequisites are in `global-requirements-defender.md`.

• **03-implementation.md** — the facilitator + student runbook: Lab 0 readiness plus the five guided labs, with named objects and pass/fail gates.

• **05-testing.md** — environment validation, the per-attendee knowledge-check, and the capstone demonstration.

• **06-troubleshooting.md** — symptom → cause → fix for the common enablement kit and MDA issues.

• **07-best-practices.md** — how to run the enablement kit and operate MDA well, plus where to go next in the catalog.

• **08-deliverables-summary.md** — this file.

• **global-requirements-defender.md** — the category-level prerequisites (shared across all Defender solutions).

Lab-helper assets (scripts/)

• **WSMCA-Discovery.ps1** — read-only discovery (Microsoft Graph): MDA alert count, connected-app signal, and an OAuth app count/inventory summary for Lab 0. Makes no changes.

• **WSMCA-Validate.ps1** — read-only readiness validation: Graph connectivity + read scopes, MDA alert data reachable, connected-app/OAuth signal, and advanced-hunting reminder. Green = ready to teach.

• **WSMCA-Report.ps1** — read-only enablement kit summary report: MDA alerts by severity plus shadow-IT / OAuth shortlist templates for Labs 1 and 3.

• **WSMCA-LabQueries.kql** — advanced-hunting KQL pack for Labs 1, 2, 3, and 5: shadow-IT / discovery, connected-app activity, OAuth apps, and anomaly-investigation queries over `CloudAppEvents`.

• **WSMCA-OAuthApp-Lab.ps1** — read-only Lab 3 helper: enumerates OAuth service principals and consent grants with risk hints (high-privilege scopes, unverified publisher, broad consent) and prints the approval-gated revoke steps (**preview only — revokes nothing**).

Take-home runbook artifacts (produced during the labs)

• Saved hunting queries: `Discovery-ShadowIT – CloudApps Lab – Pilot`, `OAuthApps – CloudApps Lab – Pilot`, `Anomaly-Investigation – CloudApps Lab – Pilot`.

• The report-only session policy `Monitor-Downloads – CloudApps Lab – Pilot`.

• The `ShadowIT-Shortlist – CloudApps Lab – Pilot` and `OAuthRisk-Shortlist – CloudApps Lab – Pilot`.

- The `Investigation – CloudApps Lab – Pilot` findings document.

Delivered as a Word document and PDF, plus all five lab-helper scripts in a single ZIP — yours to keep and reuse.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WSMCA-Discovery.ps1

Purpose: Read-only environment discovery for the "Workshop — Microsoft Defender for Cloud Apps" enablement package. Confirms there is data to teach against: recent Defender-for-Cloud-Apps alerts (via Microsoft Graph Security), a count/summary of OAuth applications (service principals) with risk hints, and a note on connected-app signal. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WSMCA-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\WSMCA-Discovery.ps1 -OutputPath .\wsmca-discovery.json Requires the Microsoft.Graph modules. Connects with READ-ONLY delegated scopes; consent as a Security Reader / Security Administrator when prompted. EXPECTED RESULTS A console summary plus a JSON file: MDA alert count by severity, OAuth app count with a few risk hints, and reminders for the portal-only checks (Cloud Discovery, connectors). Use it in Lab 0 (readiness). .NOTES White-label. Read-only. Reports configuration/identifiers only; makes no changes. Feature/licensing facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\wsmca-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\WSMCA-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\WSMCA-Discovery.ps1 -OutputPath .\wsmca-discovery.json Requires the Microsoft.Graph modules. Connects with READ-ONLY delegated scopes; consent as a Security Reader / Security Administrator when prompted. EXPECTED RESULTS A console summary plus a JSON file: MDA alert count by severity, OAuth app count with a few risk hints, and reminders for the portal-only checks (Cloud Discovery, connectors). Use it in Lab 0 (readiness). .NOTES White-label. Read-only. Reports configuration/identifiers only; makes no changes. Feature/licensing facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\wsmca-discovery.json")

WSMCA-LabQueries.kql

WSMCA-OAuthApp-Lab.ps1

Purpose: Supports Lab 3 of the "Workshop — Microsoft Defender for Cloud Apps". It enumerates OAuth applications (service principals) and their delegated/app-role permission grants, flags risk hints (broad/high-privilege scopes, unverified publisher, many users consented), and prints the precise, approval-gated steps to disable/revoke a risky app in the portal. It requests NO write scope and performs NO revocation — keeping the workshop read-only/preview-first. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WSMCA-OAuthApp-Lab.ps1 powershell -ExecutionPolicy Bypass -File .\WSMCA-OAuthApp-Lab.ps1 -OutputPath .\wsmca-oauth.json Run as Security Reader / Security Administrator; consent the read-only scopes when prompted. EXPECTED RESULTS A ranked-ish list of OAuth apps with risk hints and a printed step-by-step for the approval-gated revoke. The facilitator uses this to build OAuthRisk-Shortlist. .NOTES White-label. Read-only / preview-first. Requests NO write scopes and revokes nothing. Revoking consent can break a legitimate integration — always approval-gated with the app owner. > [CmdletBinding()] param([string]$OutputPath = ".\wsmca-oauth.json")

How to run: powershell -ExecutionPolicy Bypass -File .\WSMCA-OAuthApp-Lab.ps1 powershell -ExecutionPolicy Bypass -File .\WSMCA-OAuthApp-Lab.ps1 -OutputPath .\wsmca-oauth.json Run as Security Reader / Security Administrator; consent the read-only scopes when prompted. EXPECTED

RESULTS A ranked-ish list of OAuth apps with risk hints and a printed step-by-step for the approval-gated revoke. The facilitator uses this to build OAuthRisk-Shortlist. .NOTES White-label. Read-only / preview-first. Requests NO write scopes and revokes nothing. Revoking consent can break a legitimate integration — always approval-gated with the app owner. > [CmdletBinding()] param([string]$OutputPath = ".\wsmca-oauth.json")

WSMCA-Report.ps1

Purpose: Produces a compact, read-only summary for the end of the Defender for Cloud Apps workshop: a count of Defender-for-Cloud-Apps alerts by severity/status, the most recent alerts, and fill-in templates for the "ShadowIT-Shortlist - CloudApps Lab - Pilot" and "OAuthRisk-Shortlist - CloudApps Lab - Pilot" (authored in the portal during the labs). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WSMCA-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WSMCA-Report.ps1 -OutputPath .\wsmca-report.html Run as Security Reader / Security Administrator; consent read-only scopes when prompted. EXPECTED RESULTS A console summary and an HTML report file. The shortlist sections are templates because app-sanctioning and consent-revocation decisions belong to the customer's security/business functions. .NOTES White-label. Read-only. Makes no changes. Licensing/preview facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\wsmca-report.html")

How to run: powershell -ExecutionPolicy Bypass -File .\WSMCA-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WSMCA-Report.ps1 -OutputPath .\wsmca-report.html Run as Security Reader / Security Administrator; consent read-only scopes when prompted. EXPECTED RESULTS A console summary and an HTML report file. The shortlist sections are templates because app-sanctioning and consent-revocation decisions belong to the customer's security/business functions. .NOTES White-label. Read-only. Makes no changes. Licensing/preview facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\wsmca-report.html")

WSMCA-Validate.ps1

Purpose: Confirms the environment is ready to teach the "Workshop — Microsoft Defender for Cloud Apps" labs BEFORE attendees arrive. Checks (all read-only): Microsoft Graph connectivity and the required READ scopes, that MDA alert data is reachable, that OAuth app data is reachable, and reminds the facilitator of the portal-only checks (Cloud Discovery, connectors, app governance, advanced hunting). Prints a PASS/REVIEW table. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WSMCA-Validate.ps1 Run as Security Reader / Security Administrator; consent read-only scopes when prompted. EXPECTED RESULTS A PASS/REVIEW line per check. All PASS = ready to run Lab 0 onward; any REVIEW points at the matching item in 02-requirements.md. .NOTES White-label. Read-only. Makes no changes. Licensing/preview facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\WSMCA-Validate.ps1 Run as Security Reader / Security Administrator; consent read-only scopes when prompted. EXPECTED RESULTS A PASS/REVIEW line per check. All PASS = ready to run Lab 0 onward; any REVIEW points at the matching item in 02-requirements.md. .NOTES White-label. Read-only. Makes no changes. Licensing/preview facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param()